



Savitribai Phule Pune University (Formerly University of Pune)

T.Y.B.Sc.(Computer Applications)/ T.Y.B.C.A. – Semester-V

(Faculty of Science and Technology)

(For Colleges Affiliated to Savitribai Phule Pune University)

**Choice Based Credit System (CBCS) Syllabus Under
National Education Policy (NEP)**

To be implemented from Academic Year 2026-2027

Title of the Course: B.C.A./ B.Sc.(CA)



Savitribai Phule Pune University
(Formerly University of Pune)

T.Y.B.Sc. (Computer Applications) / T.Y.B.C.A.

Semester-V

WORKBOOK

Course Code: CA – 303 MJP

**Course Title: Lab Course on CA-303 MJP:
Fundamentals of Artificial Intelligence**

Student Name			
College Name			
Roll No.		Division	
Exam Seat No.		Academic Year	

From the Chairman's Desk

It gives me great pleasure to present this workbook prepared by the Board of studies in Computer Applications.

The workbook has been prepared with the objectives of bringing uniformity in the implementation of lab assignments across all affiliated colleges, acting as a ready reference for both fast and slow learners, and facilitating continuous assessment using clearly defined rubrics.

The workbook provides, for each of the assignments, the aims, prerequisites, related theoretical concepts with suitable examples wherever necessary, guidelines for the faculty/lab administrator, instructions for the students to perform assignments, and a set of exercises divided into three sets.

I am thankful to the Chairman of this course and the entire team of editors. I am also thankful to the reviewers and members of BOS, **Dr. A.B. Nimbalkar** and **Dr. Razzak Sayyad**. I thank all members of BOS and everyone who has contributed directly or indirectly to the preparation of the workbook.

Constructive criticism is welcome and should be communicated to the Chairman of the Course and overall coordinator, **Dr. A. B. Nimbalkar**. Affiliated colleges are requested to collect feedback from the students for further improvements.

I am thankful to Hon. Vice Chancellor of Savitribai Phule Pune University **Prof. Dr. Suresh Gosavi**, Pro Vice Chancellor **Dr. Parag Kalkar**, and the Dean of Faculty of Science and Technology **Prof. Dr. Pramod D. Patil** for their support and guidance.

Prof. Dr. S. S. Sane

Chairman, BOS in Computer
Applications Savitribai Phule Pune
University, Pune

Editors:

Asst.Prof Shital Shahaji Jadhav	Sinhgad College of Science Ambegaon(BK), Pune
Dr. Jayshree Kamble	KES' Pratibha College of Commerce and Computer Studies Chinchwad
Asst.Prof Madhuri Gandhi	KES' Pratibha College of Commerce and Computer Studies Chinchwad
Asst.Prof Alka Badan Mhetre	Raimata Jijau Shikshan Prasark Mandal's ACS College, Landewadi, Bhoshari
Asst.Prof Rutuja Ganesh More	JSPM's Jaywantrao Sawant College of Engineering, Hadapsar, Pune

Reviewed By:

- **Dr. A. B. Nimbalkar**
Assistant Professor,
PDEA"s Waghire College, Pune.
- **Dr. Razzak Sayyad**
Assistant Professor,
Ahmednagar College, Ahilyanagar.

Introduction:

1. About the work book:

This workbook is intended to be used by T.Y.B.Sc. (CA) and T.Y.B.C.A students for the BCA-Fundamentals of AI Laboratory Assignments in Semester–V. This workbook is designed by considering all the practical concepts / topics mentioned in syllabus.

2. The objectives of this workbook are:

- 1) Defining the scope of the course.
- 2) To bring uniformity in the practical conduction and implementation in all colleges affiliated to SPPU.
- 3) To have continuous assessment of the course and students.
- 4) Providing ready reference for the students during practical implementation.
- 5) Provide more options to students so that they can have good practice before facing the examination.
- 6) Catering to the demand of slow and fast learners and accordingly providing the practice assignments to them.

3. How to use this workbook:

- 1) The Fundamentals of Artificial Intelligence is divided into five assignments.
- 2) It is mandatory for students to complete the SET A and SET B .

4. Instructions to the students

- 1) Students are expected to carry this lab book every time they come to the lab for practical.
- 2) Students should prepare oneself beforehand for the Assignment by reading the relevant material.
- 3) Instructor will specify which problems to solve in the lab during the allotted slot and student should complete them and get verified by the instructor. However student should spend additional hours in Lab and at home to cover as many problems as possible given in this work book.
- 4) Students will be assessed for each exercise on a scale from 0 to 5.

Not done	0
Incomplete	1
Late Complete	2
Needs improvement	3
Complete	4
Well Done	5

5. Guidelines for Instructors

- 1) Explain the assignment and related concepts in around ten minutes using white board if required or by demonstrating on Projector.
- 2) You should evaluate each assignment carried out by a student on a scale of 5 as specified above by ticking appropriate box.
- 3) The value should also been written on assignment completion page of the respective Lab course.

6. Guidelines for Lab administrator

You have to ensure appropriate hardware and software is made available to each student.

The operating system and software requirements on server side and also client-side areas given below:

1) Server and Client Side - (Operating System) Linux/Windows

2) Programming Languages & Platforms

- Python 3.8 + (Primary language)
- Anaconda / Miniconda
- Jupyter Notebook / Google Colab

Table of Contents

Assignment No.	Title	Page No.
1	Assignments I : Foundations of Artificial Intelligence using Python	10-17
2	Assignment II : Problems, Problem Spaces and Search	18-22
3	Assignment III : Game Theory	23-28
4	Assignment VI : Machine Learning - Introduction to Data Handling	29-36
5	Assignment V: Machine Learning - Supervised Learning	37-42

Assignment Completion Sheet

	Assignments			
Sr. No.	Assignment Name	Marks (out of 10)	Date	Instructor Sign
1	Assignments I :Foundations of Artificial Intelligence using Python			
	Assignment II : Problems, Problem Spaces and Search			
3	Assignment III : Game Theory			
4	Assignment V : Machine Learning - Introduction to Data Handling			
5	Assignment VI : Machine Learning - Supervised Learning			
Total (Out of 50)				
Total (Out of 15)				

Batch -Incharge

CERTIFICATE

This is to certify that Mr./Ms. _____ Has
successfully completed and submitted the course work for Lab Course CA- 303 MJP:
Fundamentals of AI, T.Y.B.Sc.(C.A)/T.Y.B.C.A SEM-V prescribed by Savitribai Phule
,Pune University during academic year and has scored marks _____out of 15 Marks..

Batch -Incharge

H.O. D / Coordinator

Internal Examiner

External Examiner

ASSIGNMENT I: Foundations of Artificial Intelligence using Python

Objective:

1. To strengthen Python programming skills required for AI development.
2. To apply logical problem-solving techniques using Python.
3. To implement basic intelligent agent models and understands agent behavior.

Basic Theory

1. Introduction to Python in Artificial Intelligence

Python is a high-level, interpreted programming language widely used in Artificial Intelligence due to its simple syntax and powerful capabilities. It supports structured programming, modular programming using functions, and efficient data handling using built-in data structures.

Python helps in implementing intelligent systems, decision-making models, and AI agents.

Python Installation

Python can be installed from the official website:

<https://www.python.org/downloads/>

Steps for Installation:

1. Visit the official Python website and download the latest stable version suitable for your operating system (Windows/Linux/macOS).
2. Run the installer and select the option “Add Python to PATH” before clicking Install.
3. Complete the installation process.

Sample Program

```
a = 10  
b = 5
```

```
print("Addition:", a + b)  
print("Multiplication:", a * b)
```

2. Basic Python Concepts

(a) Variables and Data Types

Variables are used to store data. Common data types in Python include:

- int (Integer)
- float (Decimal)
- str (String)
- bool (Boolean)

Example

```
name = "AI System"  
version = 1.0  
active = True
```

```
print(name)  
print(version)  
print(active)
```

(b) Operators

Operators are used to perform operations on variables.

Types:

- Arithmetic Operators (+, -, *, /, %)
- Relational Operators (>, <, ==)
- Logical Operators (and, or, not)

Example

```
x = 12  
y = 4
```

```
print("Division:", x / y)  
print("Is x greater than y?", x > y)
```

3. Control Structures

Control structures control the flow of execution.

(a) Conditional Statement (if–else)

Used for decision-making.

```
marks = 65
```

```
if marks >= 50:  
    print("Pass")  
else:  
    print("Fail")
```

(b) Loops

Loops are used to repeat instructions.

For Loop

```
for i in range(1, 4):  
    print("Iteration:", i)
```

While Loop

```
count = 1  
while count <= 3:  
    print("Count:", count)  
    count += 1
```

4. Data Structures in Python

Data structures are used to store and organize data efficiently. They are very important in AI systems for storing datasets, environment states, and decision logic.

(a) List

A list is an ordered and changeable collection of elements. It supports indexing, slicing, iteration, and various methods.

Example

```
numbers = [5, 10, 15, 20]
```

```
print("First element:", numbers[0])  
print("Slice:", numbers[1:3])
```

```
for num in numbers:  
    print("Value:", num)
```

(b) Dictionary

A dictionary stores data in key–value pairs.
It is useful for representing structured information.

Example

```
device = {  
    "name": "Sensor1",  
    "status": "Active",  
    "battery": 85  
}
```

```
print("Device Name:", device["name"])  
print("Battery Level:", device["battery"])
```

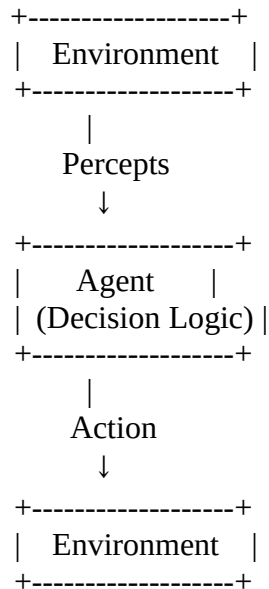
5. Introduction to Intelligent Agents

An Intelligent Agent is an entity that:

- Perceives its environment
- Makes decisions
- Performs actions

Agents aim to behave rationally by selecting actions that maximize performance.

Structure of an Intelligent Agent

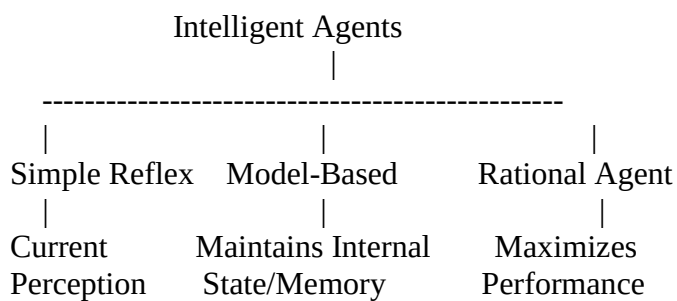


Explanation:

The agent receives input (percepts), processes it, and performs actions that affect the environment.

6. Types of Intelligent Agents

Diagram: Classification of Agents



(a) Simple Reflex Agent

Acts only on the current perception using condition-action rules.

Example

```
temperature = 32
```

```
if temperature > 30:
    print("Activate Cooling System")
else:
    print("Cooling Not Required")
```

(b) Model-Based Agent

Maintains internal memory and uses previous state information.

Example

```
previous_signal = "Red"
current_signal = "Green"
```

```
if current_signal != previous_signal:
    print("Signal Changed")
else:
    print("Signal Same")
```

```
previous_signal = current_signal
```

(c) Rational Agent

A rational agent selects the best action to maximize performance.

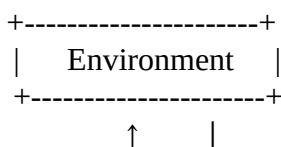
Example

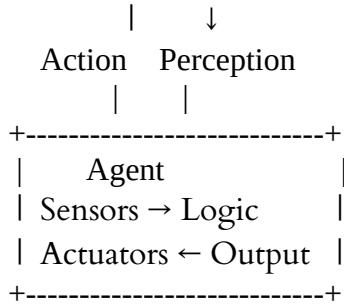
```
actions = {"Study": 9, "Play": 6, "Sleep": 7}
```

```
best_action = max(actions, key=actions.get)
print("Selected Action:", best_action)
```

7. Agent – Environment Interaction

Diagram: Agent Architecture Overview





Explanation:

- Sensors collect percepts from the environment
- Agent processes input using internal logic
- Actuators perform actions
- Interaction continues in a cycle

8. Perception – Decision – Action Cycle

All intelligent systems follow three main steps:

1. Perception (Input)
2. Decision (Processing)
3. Action (Output)

Example

```
light = input("Enter light level (low/high): ")
```

```
if light == "low":
```

```
    action = "Turn ON Light"
```

```
else:
```

```
    action = "Turn OFF Light"
```

```
print("Action:", action)
```


Lab Assignments

SET A

1. Write a Python program to print “Welcome to Artificial Intelligence”.
2. Write a Python program to take two numbers as input and display their sum, difference, product and division.
3. Write a Python program that takes employee details (name, employee number, salary) as input and displays them in a formatted manner.
4. Develop a program to perform different **list methods** such as add (append), extend, insert, remove and delete elements, and display the updated list after each operation.
5. Write a Python program to show proper indentation using if statement.

SET B

1. Write a Python program to manage multiple student records using a dictionary and determine the topper based on percentage.
2. Write a Python program using a user defined function to calculate interest amount using simple interest method and compound interest method and find the difference of interest amount between the two methods.
3. Write a Python program to simulate a simple reflex agent for fan control.
4. Create a simple chatbot using predefined rules to respond to user inputs.

SET C

1. Design a Model-Based Agent that maintains an internal state of room cleanliness and decides actions based on both current perception and stored memory.
2. Develop a Python program to demonstrate the Structure of an Intelligent Agent (Perception → Decision → Action) by simulating an agent operating in a simple environment.

Assignment Evaluation

0: Not Done []

2: Late Complete []

4: Complete []

1: Incomplete []

3: Needs Improvement []

5: Well done []

Signature of Teacher

ASSIGNMENT II: Problems, Problem Space and Search

Objective:

The objective of this assignment is to understand Artificial Intelligence problem-solving techniques using problem representation, problem space formulation and search strategies. Students will learn how AI systems represent problems, explore possible states and find optimal solutions using different search algorithms.

What is Artificial Intelligence Problem Solving?

In Artificial Intelligence (AI), problem solving is the process of finding a sequence of actions that leads from an initial state to a goal state.

An AI problem consists of:

- **Initial State**
- **Goal State**
- **Actions**
- **Path / Solution** AI solves problems by exploring different possible states using search techniques.

1. What is a Problem in AI?

A problem in AI is defined as a task that can be solved by searching through a set of possible states.

Example Problems:

- Water Jug Problem
- Missionaries and Cannibals
- 8-Puzzle Problem
- Tic-Tac-Toe
- Tower of Hanoi

2. Problem Space

Definition:

A problem space is the complete set of all possible states reachable from the initial state.

It includes:

- Initial State

- All Possible Intermediate States
- Goal State

Example: Water Jug Problem

If Jug1 = 4L and Jug2 = 3L

States can be:

(0,0)
 (4,0)
 (4,3)
 (1,3)
 (1,0)
 (0,1)
 (4,1)
 (2,3)

Each pair represents (Jug1, Jug2).

3. State Space Representation

A state space is represented using:

- Nodes $\rightarrow$ States
- Edges $\rightarrow$ Actions
- Path $\rightarrow$ Sequence of moves

AI explores this state space using search algorithms.

4. Search Techniques in AI

Search techniques are classified into:

A) Uninformed Search (Blind Search)

These methods do not use domain knowledge.

1. **Breadth First Search (BFS)**
2. **Depth First Search (DFS)**
3. **Uniform Cost Search**

B) Informed Search (Heuristic Search)

These methods use heuristics (intelligent guidance).

1. **Hill Climbing**
2. **Best First Search**
3. **A* Search**

5. Breadth First Search (BFS)

Definition: BFS explores all nodes level by level using a **Queue (FIFO)**.

Characteristics:

- Complete
- Optimal (for equal cost)
- Uses more memory

Applications:

- Shortest Path Problems
- Missionaries & Cannibals
- Water Jug

6. Depth First Search (DFS)

Definition: DFS explores one branch completely before backtracking using a **Stack (LIFO)**.

Characteristics:

- Less memory
- May not give optimal solution
- Faster in deep search

7. Hill Climbing

Definition: Hill Climbing is a local search algorithm that moves toward better neighboring states.

Characteristics:

- Greedy approach
- May get stuck in local maxima

Uses heuristic function

8. Comparison of Search Techniques

Feature	BFS	DFS	Hill Climbing
Data Structure	Queue	Stack	None
Complete	Yes	No	No
Optimal	Yes	No	No
Memory Usage	High	Low	Very Low
Uses Heuristic	No	No	Yes

9. Problem Formulation in AI

A well-defined problem consists of:

1. **Initial State**
2. **Successor Function**
3. **Goal Test**
4. **Path Cost**

10. Applications of Search in Real Life

- Google Maps (Shortest Path)
- Robotics Path Planning
- Game Playing (Chess, Tic-Tac-Toe)
- Puzzle Solving
- Route Optimization

Lab Assignments

SET A

1. Write a Python program to solve Water Jug Problem.
2. Implement Tic-Tac-Toe game using Python.
3. Solve Tower of Hanoi using recursion.
4. Implement Monkey Banana Problem.
5. Implement Hill Climbing Algorithm.

SET B

1. Solve Missionaries and Cannibals using DFS.
2. Solve Missionaries and Cannibals using BFS.
3. Solve Blocks World Problem.
4. Compare BFS and DFS performance.

SET C

1. Draw State Space diagram for Water Jug Problem.
2. Explain problem formulation of 8-Puzzle.
3. Compare Uninformed and Informed Search.
4. Write advantages and disadvantages of Hill Climbing.

Assignment Evaluation

0: Not Done []

2: Late Complete []

4: Complete []

1: Incomplete []

3: Needs Improvement []

5: Well done []

Signature of Teacher

ASSIGNMENT III: Game Theory

Objective: To study and implement basic concepts of Game Theory for decision-making in Artificial Intelligence.

What is Game Theory?

Game theory is a mathematical framework for analyzing strategic interactions where the outcome depends on the actions of multiple decision-makers (players). The theory was developed in the mid-20th century by pioneers like John von Neumann and John Nash.

Example: Prisoner's Dilemma (Two-Player Game)

Problem Statement

Two players can either:

- **C** → Cooperate
- **D** → Defect

The payoff matrix is:

Player A / Player B	C	D
C	(3,3)	(0,5)
D	(5,0)	(1,1)

1. If both cooperate → reward 3 each
 2. If one defects → defector gets 5, cooperator gets 0
 3. If both defect → both get 1
-

Python Code (Small & Exam Ready)

Prisoner's Dilemma - Simple Game Theory Example

Payoff Matrix

```
payoff = {  
    ('C', 'C'): (3, 3),  
    ('C', 'D'): (0, 5),  
    ('D', 'C'): (5, 0),  
    ('D', 'D'): (1, 1)  
}
```

Players choose strategy

```
player1 = input("Enter strategy for Player 1 (C/D): ").upper()
```

```
player2 = input("Enter strategy for Player 2 (C/D): ").upper()
```

Get payoff

```
result = payoff.get((player1, player2))
```

```
if result:
```

```
    print("Player 1 Payoff:", result[0])
```

```
    print("Player 2 Payoff:", result[1])
```

```
else:
```

```
    print("Invalid Strategy! Choose C or D.")
```

Key Concepts:

Players: Individuals or entities involved in the game.

Strategies: Plans of action chosen by players to achieve the best outcome.

Payoffs: Rewards or penalties resulting from the combination of players' strategies.

Equilibria: Stable states where no player can benefit by unilaterally changing their strategy

Common Types of Games:

Cooperative vs. Non-Cooperative: In cooperative games, players can form alliances and share payoffs; in non-cooperative games, players act independently.

Cooperative Game Example

Players cooperate and share total profit equally.

Scenario

If both cooperate and invest together:

- Total profit = 10
- Each gets = 5

If they don't cooperate:

- Each gets only 2

Cooperative Game Example

```
def cooperative_game(cooperate):  
    if cooperate:  
        total_profit = 10  
        share = total_profit / 2  
        return share, share  
    else:  
        return 2, 2  
  
choice = input("Do players cooperate? (yes/no): ").lower()  
  
if choice == "yes":  
    p1, p2 = cooperative_game(True)  
else:
```

```

p1, p2 = cooperative_game(False)
print("Player 1 Payoff:", p1)
print("Player 2 Payoff:", p2)

```

Non - Cooperative Game Example

Two companies decide whether to:

- **Invest (I)**
- **Not Invest (N)**

Payoff Matrix

Player A / Player B	I	N
I	(4,4)	(1,5)
N	(5,1)	(2,2)

Non-Cooperative Game Example

```

payoff = {
    ('I', 'I'): (4, 4),
    ('I', 'N'): (1, 5),
    ('N', 'I'): (5, 1),
    ('N', 'N'): (2, 2)
}

player1 = input("Player 1 strategy (I/N): ").upper()
player2 = input("Player 2 strategy (I/N): ").upper()
result = payoff.get((player1, player2))

if result:
    print("Player 1 Payoff:", result[0])
    print("Player 2 Payoff:", result[1])

```

else:

```
print("Invalid Strategy!")
```

Zero-Sum vs. Non-Zero-Sum: In zero-sum games, one player's gain is another's loss; in non-zero-sum games, all players can gain or lose together.

How Game Theory Can Enhance AI Decision-Making?

Game theory provides a robust mathematical framework that can significantly enhance the decision-making capabilities of AI systems. By incorporating game-theoretic principles, AI can handle complex interactions between multiple agents more effectively. Here's how game theory can enhance AI decision-making:

1. Strategic Thinking

Anticipating Actions: Game theory allows AI agents to predict and anticipate the actions of other agents, whether they are other AI systems or human participants.

Strategic Planning: By understanding the potential moves of others, AI can devise more sophisticated strategies that consider the likely responses of other agents.

2. Optimization of Strategies

Maximizing Payoffs: Game-theoretic models help AI agents identify strategies that maximize their payoffs in various scenarios, ensuring they make the most advantageous decisions.

Balancing Trade-offs: In situations involving multiple objectives, game theory aids in balancing trade-offs to achieve the best overall outcome.

3. Equilibrium Analysis

Stable Solutions: Game theory provides tools such as Nash equilibrium, where AI agents can find stable strategies that no agent has an incentive to deviate from, leading to predictable and consistent outcomes.

Convergence to Equilibrium: AI systems can be designed to converge to equilibrium states, ensuring stable and reliable performance over time.

4. Conflict Resolution

Negotiation and Bargaining: Game theory equips AI with the ability to negotiate and reach agreements in competitive environments, facilitating cooperation and conflict resolution.

Mediation and Arbitration: In scenarios where direct competition occurs, game theory helps AI mediate and arbitrate to find mutually beneficial solutions.

5. Handling Uncertainty and Dynamics

Adaptive Strategies: Game-theoretic models enable AI to adapt its strategies in dynamic environments where conditions and agent behaviors change over time.

Probabilistic Reasoning: By incorporating probabilities, game theory helps AI deal with uncertainty and make informed decisions based on likely outcomes.

Lab Assignments

SET A

1. Write a Python program to solve tic-tac-toe problem.
2. Write a Python program to implement the Alpha–Beta pruning with depth limit and heuristic

SET B

1. Write a Python program to implement Mini-Max Algorithm.
2. Write a Python program to simulate n-Queens problem.

SET C

1. Write a Program to Implement Alpha-Beta Pruning using Python
2. To implement and simulate a simple Stochastic Game using python

Assignment Evaluation

0: Not Done []

2: Late Complete []

4: Complete []

1: Incomplete []

3: Needs Improvement []

5: Well done []

Signature of Teacher

ASSIGNMENT IV: Machine Learning Introduction to Data Handling

Objective:

The objective is to efficiently handle and load , prepare data, including cleaning and processing string-type fields, load the dataset, divide it into training and testing sets, train the model on the training portion, evaluate its performance on the test data, and ultimately use the trained model to make accurate predictions on new inputs and also make data visualization.

Understanding

What is Machine Learning?

Machine learning (ML) is a subset of artificial intelligence (AI) that enables systems to automatically learn, improve, and make predictions from data without being explicitly programmed. It uses statistical algorithms to uncover patterns in data, allowing computers to learn from experience and make decisions with minimal human intervention

Set Up Machine Learning Environment

Before you can run ML code, you need Python and some libraries installed.

Install Python

- Make sure Python 3.8+ is installed. Check by running:
Python --version
- If not installed, download from python.org
- Recommended IDE: Jupyter Notebook , Python Shell

Installation of Required Python ML libraries:

Open terminal / command prompt and run:

```
pip install numpy pandas matplotlib scikit-learn
```

Libraries used in Python for Machine Learning practicals

1. NumPy (numpy) – Numerical Python

Purpose: Perform mathematical operations on numbers and arrays efficiently. Handles arrays and matrices (like Excel sheets but faster). Provides mathematical functions like sum, mean, standard deviation, etc. Used internally by many ML libraries.

Key Use in ML: Features and labels are often stored as NumPy arrays for fast computation

2. Pandas (pandas) – Data Handling Library

Purpose: Work with tabular data (rows and columns) easily. Main object: DataFrame (like a spreadsheet or SQL table). Can read/write CSV, Excel, SQL databases. Provides functions to explore, clean, and manipulate data.

Key Use in ML: Preprocessing data before feeding it to models.

3. Matplotlib – Visualization Libraries

Purpose: Plot graphs to understand data trends. Basic plotting library. Can plot line graphs, scatter plots, histograms, etc.

Key Use in ML: Helps to visualize relationships between features and target before modeling.

4. Scikit-learn (sklearn) – Machine Learning Library

Purpose: Main Python library for building ML models. Provides algorithms: Linear Regression, Logistic Regression, Decision Trees, Random Forests, SVM, KMeans, etc. Tools for data splitting, preprocessing, and evaluation.

Key Use in ML: Build, train, predict, and evaluate machine-learning models easily.

Importing Libraries

Before coding, import required libraries like following:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
```

Explanation: train_test_split : split dataset into training and testing , LinearRegression : a basic ML algorithm for prediction

What is a Dataset?

A dataset is simply a collection of data that we use to train a Machine Learning model. Just like humans learn from experience, a machine learns from examples stored in a dataset. A dataset is usually arranged in rows and columns, similar to an Excel sheet.

A typical dataset has :

1. Features (Input Variables)

We give these inputs to the ML model. They help the model make predictions. Also called independent variables or X values. Example: Hours_Studied , Years_Experience, Age, Weight

2. Target (Output Variable)

This is the answer the model tries to predict. Also called dependent variable or y value.

Example: Scores, Salary, Fruit Type, Pass/Fail, Fitness Level

Example of an array: [Dataset]

[99,86,87,88,111,86,103,87,94,78,77,85,86]

Example : House Price Prediction Dataset (Regression)

Goal: Predict house price

Area_sqft	Bedrooms	Age	Price
1000	2	5	50,00,000
1500	3	10	75,00,000
2000	4	3	90,00,000

Features (Input): Area_sqft, Bedrooms, Age,

Target (Output): Price

Used For: Linear Regression / Random Forest Regression

Data Structure in Pandas : DataFrame

A DataFrame is a two-dimensional, size-mutable and potentially heterogeneous tabular data structure with labeled axes (rows and columns).

Creating a DataFrame from a List

A simple way to create a DataFrame is by using a single list. Pandas automatically assigns index values to the rows when you pass a list.

Sample Example:01

```
import pandas as pd
lst = ['abc', 'For', 'xyz', 'is', 'portal', 'for', 'pqr']
df = pd.DataFrame(lst)
print(df)
```

Creating a DataFrame from a List of Dictionaries

We can also create a dataframe using List of Dictionaries. It represents data where each dictionary corresponds to a row.

Sample Example : 02

```
import pandas as pd
dict = {'name': ["aparna", "pankaj", "sudhir", "Geeku"],
        'degree': ["MBA", "BCA", "M.Tech", "MBA"],
        'score': [90, 40, 80, 98]}
df = pd.DataFrame(dict)
print(df)
```

Step by Step Data Handling

1. Load Your Dataset

You need data to train your ML model. You can use CSV files (.csv) , Excel files (.xlsx). Built-in datasets from scikit-learn

Sample student_scores.csv

Hours, Scores

2.5, 21

5.1, 47

3.2, 27

8.5, 75

3.5, 30

1.5, 20

Sample Example :03 Dataset Loading using CSV file

```
import pandas as pd
data = pd.read_csv("student_scores.csv")
print(data.head()) # Shows first 5 rows
```

Sample Example 04 using scikit-learn dataset:

```

from sklearn.datasets import load_iris
iris = load_iris()
X = iris.data
y = iris.target
print(X[:5])
print(y[:5])

```

Convert a built-in dataset (like Iris) to DataFrame

Many ML datasets (Iris, Wine, Digits) come from sklearn.

Sample Example 05

```

from sklearn.datasets import load_iris
import pandas as pd
iris = load_iris()
# Convert to DataFrame
df = pd.DataFrame(iris.data, columns=iris.feature_names)
# Add target column
df['target'] = iris.target
print(df.head())

```

2.Data Exploration(Always explore your dataset before using it.) Why?

Understand the features and target. Check for missing data. Visualize relationships.

Basic commands:

Sr	Function	Description	Example
1	pd.read_csv()	Load a CSV file	data = pd.read_csv("data.csv")
2	head()	Show first 5 rows	data.head()
3	tail()	Show last 5 rows	data.tail()
4	shape	Show rows & columns count	data.shape
5	columns	Show column names	data.columns
6	info()	Show data types & memory	data.info()
7	describe()	Show statistics summary	data.describe()
8	isnull()	Check missing values	data.isnull()
9	isnull().sum()	Count missing values	data.isnull().sum()
10	dropna()	Remove missing rows	data.dropna()
11	fillna()	Fill missing values	data.fillna(0)
12	duplicated()	Check duplicate rows	data.duplicated()
13	drop_duplicates()	Remove duplicate rows	data.drop_duplicates()
14	sort_values()	Sort dataset	data.sort_values("Age")
15	value_counts()	Count category values	data["Gender"].value_counts()
16	unique()	Show unique values	data["City"].unique()
17	nunique()	Count unique values	data["City"].nunique()
18	loc[]	Select by label	data.loc[0]
19	iloc[]	Select by index	data.iloc[0:5]
20	corr()	Correlation between columns	data.corr()
21	rename()	Rename columns	data.rename(columns={"A":"B"})
22	drop()	Remove row/column	data.drop("Age", axis=1)

23	sample()	Select random rows	data.sample(5)
24	merge()	Join two datasets	pd.merge(df1, df2, on="ID")
25	groupby()	Group data	data.groupby("Gender").mean()

3. Visualization

Data visualization is a graphical representation of data and information. With the help of data visualization, we can see how the data looks like and what kind of correlation is held by the attributes of the data

Scatter Plot :

A **scatter plot** is a type of graph used to display the relationship between **two numerical variables**. It shows data points as **dots** on a graph.

Sample Example 06 : Scatter plot and Line Plot

```
import matplotlib.pyplot as plt
# Sample Data
x = [1, 2, 3, 4, 5]
y = [2, 4, 5, 4, 5]
plt.scatter(x, y, label="Data Points") # Scatter Plot
plt.plot(x, y, label="Line", marker='o') # Line Plot
plt.title("Scatter Plot with Line")      # Assign Title
```

4. Preparing Data for ML

Split features and target

```
X = data[['Hours_Studied']] # Feature(s)
```

```
y = data['Scores']          # Target
```

Split data into training and testing

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

Explanation: test_size=0.2 → 20% of data is for testing. random_state → ensures reproducibility.

5. Choosing and Training a Model

Example: **Linear Regression**

```
model = LinearRegression()
```

```
model.fit(X_train, y_train)
```

- fit() → ML model “learns” from training data.
- For beginners, start with **Linear Regression** → **Logistic Regression** → **Decision Tree**.

6. Making Predictions

```
y_pred = model.predict(X_test)
```

```
print(y_pred)
```

Pandas Str Methods for Handling Dataset

str Method	Description	Example Code	Expected Output
str.upper()	Converts to uppercase	df["Name"].str.upper()	alice → ALICE
str.lower()	Converts to lowercase	df["Name"].str.lower()	BOB → bob
str.title()	First letter capital	df["Name"].str.title()	charlie → Charlie
str.len()	Length of string	df["Name"].str.len()	Charlie → 7
str.contains()	Checks pattern	df[df["City"].str.contains("i")]	Mumbai, Kochi
str.startswith()	Checks prefix	df[df["City"].str.startswith("C")]	Chennai
str.endswith()	Checks suffix	df[df["City"].str.endswith("i")]	Kochi
str.replace()	Replace a substring	df["Product"].str.replace("Mobile","Phone")	Mobile-01 → Phone-01
str.split()	Split by delimiter	df["Full"].str.split(" ")	[John, Smith]
str.strip()	Remove spaces	df["Text"].str.strip()	" hello " → "hello"
str.extract()	Extract regex match	df["Code"].str.extract("(\\d+)")	A-101 → 101
str.findall()	Find all regex matches	df["Words"].str.findall("at")	[at, at, at]
str.isnumeric()	Check if numeric	df["Value"].str.isnumeric()	"123" → True
str.isalpha()	Check alphabetic	df["Value"].str.isalpha()	"abc" → True
str.isalnum()	Alpha-numeric check	df["Value"].str.isalnum()	"abc123" → True
str.pad()	Pad string left/right	df["Code"].str.pad(6, fillchar='0')	123 → 000123

Sample Example 07

```
import pandas as pd
df = pd.DataFrame({
    "Name": ["A", "B", "C", "D"],
    "Age": [21, 25, 20, 22],
    "Marks": [88, 72, 95, 60]})
# Sort by Marks descending
print(df.sort_values("Marks", ascending=False))
# Filter students with marks > 80
print(df[df["Marks"] > 80])
```

Lab Assignments

SET A

1. Write a Python program to load student.csv containing Hours and Scores of student dataset and perform basic data exploration
 - a. Display the first five rows
 - b. Print the shape of the dataset.
 - c. Show column names.
 - d. Identify missing values
 - e. Display dataset info
2. Write a Python program clean the dataset by handling missing and duplicate values use any Dataset
 - a. Check missing values
 - b. Check duplicated rows.
 - c. Remove duplicates
 - d. Store cleaned dataset in a new CSV file.
3. Write a Python program to separate features and target, and split data into train and test sets and display the shape of the dataset .Use any dataset.
4. Write a Python program to create a DataFrame which contains a person name and age ,Sort a DataFrame in ascending order of Age .

SET B

1. Write a Python program to visualize a dataset using scatter plot for sales_data.csv containing Advertising_Budget, Sales.
2. Write a Python program to load the Diabetic inbuilt dataset, convert it into a DataFrame, and display the first 5 rows and the shape of the dataset.
3. Write a Python Program to Split the Iris dataset into training and testing sets (80-20) and print the shapes of X_train and X_test.
4. Write a Python program to create dataframe which contain Employee Name, Dept, salary Display employees which satisfy following condition
 - Department = "IT"
 - Salary > 50000
 - Name contains "a"

SET C

1. Write a Python program to Plot a scatter graph of sepal length vs sepal width for the Iris dataset. Label the axes and add a title.
- 2 .Write a Python program for DataFrame of employees with Salary and Experience:
If Experience < 3 years, increase salary by 10%, else increase by 5%. and display updated DataFrame.

Assignment Evaluation

0: Not Done []

2: Late Complete []

4: Complete []

1: Incomplete []

3: Needs Improvement []

5: Well done []

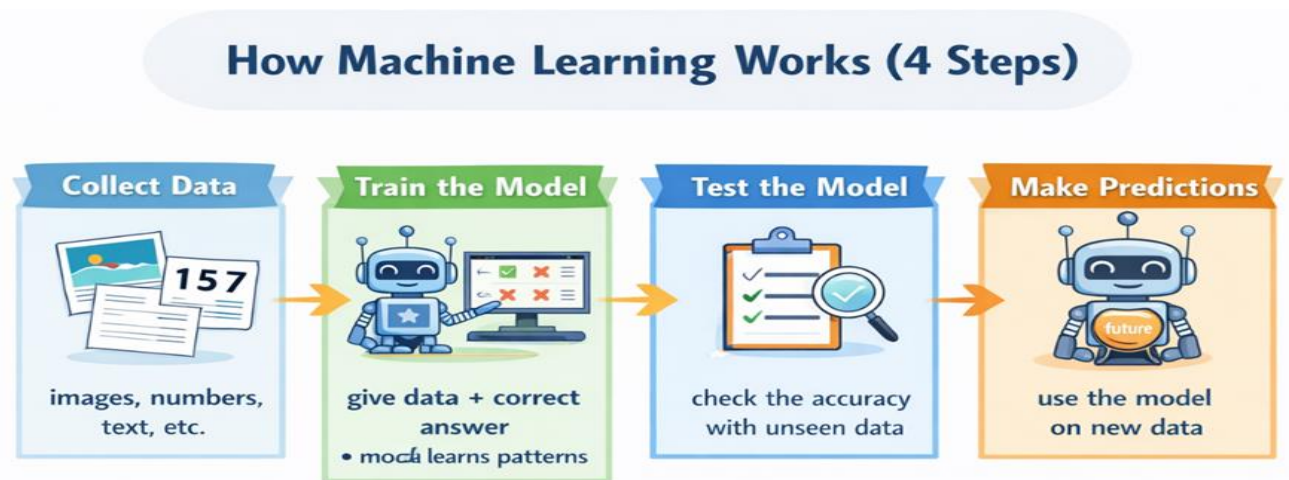
Signature of Teacher

ASSIGNMENT V: Machine Learning - Supervised Learning

Objective:

To learn a predictive mapping from input features to output labels using labeled training data, through algorithms such as linear regression, logistic regression, decision trees, random forests, enabling the model to minimize error and generalize well to unseen data and for the student to understand how these algorithms work, how to prepare data, how to evaluate models, and how to select and tune the appropriate algorithm for a given problem.

Understanding



Example : You want the computer to predict if an email is *spam*.

You do not write rules. You give it:

Email Text	Label
"Win money now"	spam
"Meeting tomorrow"	not spam

The machine learns a pattern and predicts for new emails.

Types of Machine Learning

Machine Learning is mainly divided into 3 major types:

1. Supervised Learning : Uses labeled data (input + correct answer) , Goal: Predict output

Examples: Predict marks, Spam or not spam, House price prediction

Formula : Learn from past data → Predict for new data

2. Unsupervised Learning : Uses unlabeled data (no correct answer), Goal: Find patterns, groups, structure

Examples: Group customers, Market segmentation, Topic clustering

Formula: Model finds hidden patterns by itself.

It has two types of tasks:

1. **Regression (predicting continuous values)**
2. **Classification (predicting categories)**

Below are the most commonly used supervised ML algorithms.

1. Linear Regression

Used for: Linear regression finds the best straight line that explains how one variable affects another so you can predict future values. Predicting continuous numeric values (house price, salary). Uses one input (feature) to predict the output.

Algorithm Idea: Finds the best-fit straight line: $y=mx+cy$

It helps answer questions like: “If I study more, will my grades go up?”, “If a house is bigger, will the price be higher?”, “If the temperature increases, will ice cream sales go up?”

Python Syntax :

```
from sklearn.linear_model import LinearRegression
model = LinearRegression()
model.fit(X, y)
model.predict(X_new)
```

Explanation of Each Line

```
from sklearn.linear_model import LinearRegression
```

// This imports the LinearRegression class from scikit-learn.

```
model = LinearRegression()
```

// This creates an empty linear regression model that is ready to be trained.

```
model.fit(X, y)
```

$X \rightarrow$ Input data (features)

$y \rightarrow$ Output data (labels)

// This line teaches the model the relationship between X and y. fit() = learn the pattern

```
model.predict(X_new)
```

$X_new \rightarrow$ new input values

//The model uses the learned relationship to make predictions predict() = give output

Example 1 : Predict a student's **marks** based on **hours studied**.

```
from sklearn.linear_model import LinearRegression
import numpy as np
# Data
X = np.array([[1], [2], [3], [4]]) # hours studied
y = np.array([20, 40, 60, 80]) # marks
# Create model
model = LinearRegression()
# Train model
model.fit(X, y)
# Predict marks for 5 hours
prediction = model.predict([[5]])
print("Predicted marks for 5 hours study =", prediction[0])
Expected Output example 1 : Predicted marks for 5 hours study = 100.0
```

Multiple Linear Regression?

Multiple Linear Regression is a machine-learning algorithm that predicts an output using two or more input features.

Example : 2 Predict a student's final exam marks using: Hours Studied Number of Classes Attended

```
import pandas as pd
from sklearn.linear_model import LinearRegression
# Easy dataset
data = {
    "Hours": [2, 3, 4, 5, 6],
    "Classes": [3, 4, 4, 5, 6],
    "Marks": [50, 60, 65, 70, 80] }
df = pd.DataFrame(data) #Convert data to Dataframe
# Input features
X = df[['Hours', 'Classes']]
y = df['Marks']
# Create model
model = LinearRegression()
# Train model
model.fit(X, y)
# Predict for a new student
print("Predicted Marks for 4 hours study & 5 classes:")
print(model.predict([[4, 5]])[0])
Possible Output example 2: Predicted Marks for 4 hours study & 5 classes: ≈ 67.2
```

2. Logistic Regression

Logistic Regression is a supervised ML algorithm used for classification. It answers questions like: Will it rain tomorrow? (Yes/No), Is this email spam? (Yes/No), Will a customer buy the product? (Yes/No). So instead of predicting a number (like linear regression), it predicts a probability (a value between 0 and 1).

It gives output as probability (0 to 1) and converts it into: 0 → No / Negative class, 1 → Yes

/ Positive class .It uses a linear equation (like linear regression) but Converts output into probability using the Sigmoid Function.

Example 3 : Predict whether a student will **Pass (1)** or **Fail (0)** based on **hours studied**.

```
from sklearn.linear_model import LogisticRegression
# Dataset: [Hours], [Pass/Fail]
X = [[1], [2], [3], [4], [5]]
y = [0, 0, 0, 1, 1]
model = LogisticRegression()
model.fit(X, y)
# Predict for a new student
hours = 3.5
prediction = model.predict([[hours]])
print("Prediction:", prediction[0])
```

Possible Output example 03 : Prediction: 1 (Meaning: The student is predicted to **Pass**.)

3. Decision Tree

A Decision Tree is a machine-learning algorithm that makes decisions like a flowchart.

It asks a series of questions, and based on the answers, it moves to the next step until it reaches a final decision (Yes/No or a value). Example: Is the weather good? Yes → Go outside, No → Stay inside. Works like how humans think , Follows step-by-step logic, Used for classification AND regression

Example 4 : Create a small dataset for predicting whether a child should play outside. **Your dataset must include the** following features: **Weather** → Sunny / Rainy , **Temperature** → Hot / Cold, **Play Outside** → Yes / No

```
from sklearn.tree import DecisionTreeClassifier

# Dataset: [Sunny?, Hot?]
X = [ [1, 1], # Sunny, Hot
      [1, 0], # Sunny, Cold
      [0, 1], # Rainy, Hot
      [0, 0]] # Rainy, Cold
y = ["Yes", "Yes", "No", "No"] # Play Outside?
model = DecisionTreeClassifier()
model.fit(X, y)
# ---- USER INPUT ----
sunny = int(input("Is it sunny? (1=Yes, 0=No): "))
hot = int(input("Is it hot? (1=Yes, 0=No): "))
# Prediction
prediction = model.predict([[sunny, hot]])[0]
print("\nShould you play outside? ->", prediction)
```


Output Example 4: Is it sunny? (1=Yes, 0=No): 1
Is it hot? (1=Yes, 0=No): 1
Should you play outside? -> Yes

4. Random Forest

A Random Forest is a machine learning algorithm that uses many decision trees to make a prediction. Instead of one tree → it builds a forest of many trees each tree gives its own prediction the final prediction is based on majority vote (classification) or average (regression) This makes the model more accurate and more stable than a single Decision Tree.

Example 5: Should You Play Outside?

```
from sklearn.ensemble import RandomForestClassifier
X = [ [1, 1],[1, 0],[0, 1],[0, 0],]
y = ["No", "Yes", "No", "No"]
model = RandomForestClassifier()
model.fit(X, y)
weather = int(input("Enter Weather (1 = Sunny, 0 = Rainy): "))
temp = int(input("Enter Temperature (1 = Hot, 0 = Cold): "))
prediction = model.predict([[weather, temp]])[0]
print("\nPrediction: You should", ("Play" if prediction == "Yes" else "Not Play"))
```

Sample Output Example 5: Enter Weather (1 = Sunny, 0 = Rainy): 1
Enter Temperature (1 = Hot, 0 = Cold): 0
Prediction: You should Play

Lab Assignments

SET A

1. Write a python program that predicts temperature based on the number of ice creams sold using Linear Regression Create Dataframe which contain temperature and number of icecream sold.
2. Write a Python program that predicts the price of a house using a dataset that includes Area (sq.ft) and Number of Bedrooms as input features from the user. Use Multiple Linear Regression to build the prediction model.
3. Write a python program to predict if a Person is Eligible to Vote ($\text{Age} \geq 18$) using simple Logistic Classification. Accept person age from user.
4. Write a Python program using Decision Tree Classification to predict the type of fruit (Apple or Orange) based on the following features: Weight of the fruit (in grams) ,

Texture of the fruit : Smooth = 0 , Rough = 1

SET B

1. Write a Python program to predict salary based on years of experience using Simple Linear Regression. Take years of experience as input from the user and display the predicted salary. Also draw a scatter plot and regression line.
2. Write a Python Program for Multiple Linear Regression that loads data from a CSV file to predict Sales based on Advertising Budget (TV, Radio, Newspaper).
3. Write a Python program that predicts diabetes for multiple people in one run using Logistic Regression. Use the following four input features: Fasting Sugar Level , Post-Meal Sugar Level, Age, BMI (Body Mass Index)

SET C

1. Write a Python program to develop fitness predictors using Random Forest Classification to predict whether a person is *FIT* or *UNFIT* based on: Age, Exercise Hours per Week Accept the input from user and predict for multiple people.
2. Write a Python program using Logistic Regression to detect credit card fraud using the following features: Transaction Amount, Card Age (in years), Transaction Time (0–23), Location Match Score (0–100), Online Transaction (1 = Yes, 0 = No), Train the model and predict fraud (1) or legit (0) based on user input.

Assignment Evaluation

0: Not Done []

2: Late Complete []

4: Complete []

1: Incomplete []

3: Needs Improvement []

5: Well done []

Signature of Teacher